

Project updates: what's next

The latest client feedback sets three core requirements that cannot be missed from now on. Where anything later in this course says otherwise, these updates apply.

1 Enforce the 80/20 rubric split

At least 80% of positive points · Task Completion

Did the agent produce the required artifact, follow the instructions and deliver the outcome the user asked for?

20% at most

Anything else: tool use, process, agent narration

Whenever possible, avoid process and tool use rubrics altogether. The client specifically called this out as important.

A SIMPLE RULE A criterion on **how the agent got there** belongs in the 20%.
A criterion on **what the agent actually delivered** belongs in the 80%.

When in doubt, write the Task Completion criterion instead.

TWO MORE FIXES FROM THE SAME REVIEW

- ✗ **Never write a rubric that only tests whether a file exists.**
- ✗ **Every literal a criterion references matches the actual attachment exactly.** One rubric cited `insta.tiff` when the file was `klin.png`.

2 Follow the new use case taxonomy

- The client has shared **additional categories and subcategories**, and tasks have to follow them much more closely.
- Keep every task **clearly aligned with its intended use case**, without drifting into other categories.
- Specific **tool expectations** may follow for some scenarios, to keep tasks aligned with the taxonomy.

3 The next 200 tasks are single turn

- The next block is **200 single turn tasks**, rather than multi turn.
- Several open items are still being addressed on multi turn, so the client wants to move some volume through single turn tasks.

Common Errors

The mistakes that keep costing people their tasks and their project status, and what to do instead.

Every error in this deck cites its source. Each one comes with a direct reference to the exact section of the Guidelines, the QC spec or its relevant project resource that sets the correct standard, so you can check it yourself. **Each example number is a link** to the real task where it happened.

All of the errors in this deck come from **real situations**, seen repeatedly across both attempters and reviewers. Many of them are among the most basic standards of the project, and correcting them usually takes very little extra effort.

These are not minor issues. Repeating these mistakes can directly affect your access to tasks and your ability to remain on the project.

You can go through this course superficially and risk making the same mistakes again, or use it to understand exactly what is going wrong and what needs to change. Going forward, these standards are expected to be applied consistently. **If the same fundamental issues appear again after this guidance, continued access to tasks and to the project should not be expected.**

HOW TO USE THIS DECK

Fix order is build order

The Guidelines' eight steps, and the sections whose mistakes are made at each. The earlier the step, the more a fix costs.

STEP	WHAT YOU ARE DOING	SECTIONS WHOSE MISTAKES ARE MADE HERE
1-2	Plan the task and set up the run Guidelines · 1 Planning the Agent Task · 2 Setting Up the Run	1 The Foundations Every Task Must Get Right 2 What you may not tell Model A or Model B · the input folder and opening prompt both legs share
3-4	Run Leg A, writing each follow up Guidelines · 3 Task Runner - Leg A	2 What you may not tell Model A or Model B · every follow up you send Model A
5	Write the objective rubric, rate Model A Guidelines · 5 Objective Rubrics	4 A rubric that separates two runs 5 Criteria a correct run can pass
6	Turn the Model A conversation into milestones Guidelines · 6 Milestones	3 Milestones
7	Steer Leg B to the golden, then complete it Guidelines · 7 Golden Solution - Leg B	2 What you may not tell Model A or Model B · every turn you send Model B 5 Criteria a correct run can pass · the golden passes every objective criterion 7 Closing the task · the golden itself
8	Write the subjective block, rate both models Guidelines · 8 Subjective Rubric Block	6 The subjective block
	Before you submit	7 Closing the task · the close out

The Foundations Every Task Must Get Right

Two decisions made before prompt 1 that cannot be repaired afterwards. Each one is cheap to check and expensive to get wrong.

The complexity bar

The client sets a minimum floor for each deliverable type and ranks the types by priority. Clear it while you are still designing the scenario: the artifact the opening prompt defines shapes the inputs, the workflow and the rubric that follow.

- Explainer video, P0** · several scenes driven by the task's data, not one static frame with narration
- Interactive HTML, P0** · controls that respond, state that changes; a styled static page does not qualify
- Dashboard, P0** · several linked views over data the model extracted itself
- Presentation, P1** · 10 to 15 slides carrying a structured narrative
- Designed PDF, P1** · multi page, with a layout that was designed
- CSV or structured data, P1** · multi row, derived from the media, in the target platform's schema
- Document or report, P2** · sections, tables, and figures the text refers to

Why Raising complexity is one of the client's clearest and most consistent requirements. Only the prompts and inputs reach the agent, so a deliverable no prompt asks for cannot be graded.

SOURCE
Guidelines · Complexity Bar
Guidelines · 1.3 Defining the Draft History

SEE EXAMPLE 1 ↗

The universe has to hold what the task needs

The loadout decides what the task can be, so explore the assigned universe before you draft anything. Undocumented does not mean infeasible: test the workflow before ruling it out.

- Confirm what is loaded** · which servers and data sources the session really has
- Ask the in universe agent** · it tells you what it can reach and what it cannot
- Query the data** · SQL shows the tables, relationships and edge cases the visualizers never surface
- Anchor the dates** · universe calendars hold fixed dates, so write the range you saw into the prompt
- Confirm there is enough material** · the universe and the inputs sustain every follow up, not just the first request

Why Data the server does not hold, asked for anyway, is your defect, not a model failure. QC fails a task whose primary request the available tools cannot answer.

SOURCE
Guidelines · 1.2.1 Start With the Universe
Guidelines · 4. Valid vs. Invalid Model Failures

SEE EXAMPLE 2 ↗

What you may not tell Model A or Model B

Leg A defines the task: the opening prompt and the follow ups you send Model A. Leg B is the golden run: Model B gets the same opening prompt, word for word, then the turns you send it as you steer it through the milestones. The no leak rule and the input folder are the same in both legs; what a turn may point at is not.

BOTH LEGS · EVERY TURN YOU TYPE

No turn gives the answer away. Not the opening prompt, not a Leg A follow up, not a Leg B turn: none may give Model A or Model B a value, count, position or conclusion it should derive from the inputs. A leak fails the task, however late it comes.

SOURCE
QC spec · Multi-Turn - Simulator Answer Leak
Guidelines · 7.1 What Changes from Leg A

SEE EXAMPLES [4](#) AND [5](#)

BOTH LEGS · THE OPENING PROMPT

One prompt, sent to both models. You write it once. It opens Leg A with Model A, and Leg B sends it to Model B word for word, so whatever it gives away reaches both. It keeps thresholds, rules and formats in the input files, where each model must find them.

SOURCE
Guidelines · 1.2.3 Create the Initial Prompt and the Follow-ups
Guidelines · 7.1 What Changes from Leg A

BOTH LEGS · THE INPUT FOLDER

Nothing in it previews a later turn. One folder, in the same universe for both legs, open to both models from their first turn. Deferred files ship in it from upload, neutrally named. A name or README revealing what a later turn brings or asks fails the task; a name hinting at what the file holds costs points.

SOURCE
QC spec · Input Artifacts - Deferred Asset Handling
Guidelines · 1.2.2 Select the Multimodal Inputs

SEE EXAMPLE [6](#)

WHERE THE LEGS DIFFER	LEG A ONLY · the follow ups you send Model A	LEG B ONLY · the turns you send Model B
After a miss	A punctual correction is fine when it comes up naturally in the conversation, as long as it does not compromise the 50% failure threshold.	Hint: say what is still missing, from the user's point of view, and point at the area. "A is wrong" is fine; "A is wrong, do B instead" is penalized. Naming the exact file to open is over steering: it costs points. SEE EXAMPLES 3 AND 4
A deferred file arrives	The follow up that releases it names it, and says what Model A needs from it, never the value the file exists to supply. SEE EXAMPLE 5	The turn that hands Model B the same file carries that follow up's intent, expressed the same way, so a value the follow up gave away reaches Model B too. SEE EXAMPLE 5
	SOURCE Guidelines · 1.2.3 Create the Initial Prompt and the Follow-ups QC spec · Multi-Turn - Simulator Answer Leak · Multi-Turn - Turn Structure & Dependency	SOURCE Guidelines · 7.2 Steering the Run & Hinting · 7.1 What Changes from Leg A QC spec · Multi-Turn - Simulator Answer Leak

Milestones

Written from the Leg A conversation with Model A, used in Leg B to steer Model B. Three standards cover what sits around the intent line: Modifier and Assets Delivered, the stopping rules, and one milestone per requirement on every Leg A turn.

Modifier and Assets Delivered describe the turn

Modifier is one of four, set by what the milestone's Leg A turn does to work Model A already delivered. Assets Delivered names in full each file handed over at that turn, or stays empty.

- opening** · Leg A turn 1, or a new ask touching nothing delivered
- deferred_asset** · material released into scope at this turn
- clarification_gated** · one decision the opening prompt leaves open
- revision** · something already delivered changes

Why QC grades these fields on their own, and fails the task when a deferred file handed over is missing from Assets Delivered.

SOURCE
Guidelines · 6.1 What a Milestone is Made of?
Guidelines · 6.3 Modifiers

SEE EXAMPLES [7](#), [8](#) AND [9](#)

MOST OFTEN BROKEN

Continuation criteria gate the turn

The stopping rule for a milestone: in Leg B you read it against Model B's response, and never type it to Model B. One clause per requirement of that Leg A turn, separated by pipes, each true or false on inspection, at the intent level.

Why Leg B moves to the next milestone once Model B meets every clause, and hints until then. A missing clause lets it advance with a requirement unmet; a clause the turn cannot produce holds a correct response in place.

SOURCE
Guidelines · 6.2 Writing the Milestones
QC spec · Milestones - Continuation Criteria & Assets

SEE EXAMPLES [10](#) AND [11](#)

One milestone per requirement, on every Leg A turn

Once the Model A conversation is done, convert it into milestones: one per requirement, grouped by the Leg A turn it came from. A follow up carrying three requirements produces three milestones, not one.

Why The milestone set is the only script you steer Model B with in Leg B. A Leg A turn with none leaves nothing to hold Model B to there; a requirement written twice checks one thing twice.

SOURCE
Guidelines · 6.2 Writing the Milestones
Guidelines · 6. Milestones

SEE EXAMPLES [12](#) AND [13](#)

A rubric that separates two runs

A rubric exists to tell a strong run from a weak one. Five standards keep it doing that, and a set can break any of them while still looking complete.

Negatives catch the traps

Positives are the default. Negatives are the exception: around 25% of the block, generally below 30%, each a specific, plausible failure the scenario or trajectory exposes, never one added for balance.

Why The negative slots are for attractive wrong paths the model has to resist. Present on a negative means the run did the bad thing, so it names the flaw, never the virtue.

SOURCE
Guidelines · 5.3 Positive and Negative Distribution
Guidelines · 5.2 Rubric Weights

SEE EXAMPLES [14](#) [↗] AND [15](#) [↗] MOST OFTEN BROKEN

Every ask graded, and nothing nobody asked for

Walk the prompt once per turn and tick off every ask: anything you can tick needs a criterion. In the other direction, every criterion traces back to something a turn actually asks.

Why A missing critical requirement and a criterion the prompt never called for are both Major: either one makes the grade come out wrong.

SOURCE
Guidelines · 5.7 Final Rubric Checklist
QC spec appendix · Rubric Quality Definitions · Incorrect Criteria

SEE EXAMPLES [16](#) [↗] AND [17](#) [↗]

Weight is difficulty, never importance

Price how hard a criterion is to satisfy, or for a negative, how tempting the failure is. Count the axes it exercises: source coordination, reasoning depth, modality, discovery effort. Three or more is +5, exactly two is +3, one or none is +1.

Why A requirement can be critical and still +1, because meeting it is mechanical. A weight two levels off, 1 where 5 belongs, is counted as an issue on its own.

SOURCE
Guidelines · 5.2 Rubric Weights
QC spec appendix · Rubric Quality Definitions · Incorrect Weights - Major

SEE EXAMPLE [18](#) [↗]

A rubric that separates two runs

A rubric exists to tell a strong run from a weak one. Five standards keep it doing that, and a set can break any of them while still looking complete.

At most five trajectory criteria

Trajectory rubrics: a maximum of five, a minimum of zero, and not mandatory. The Trajectory target is for what only the reasoning path shows: an identification, a dependency order, a piece of restraint. Never a named tool call.

Why Whatever the deliverable itself shows is graded on the Final Answer Artifact. Past five, the set grades the route instead of the result, and usually grades the same evidence twice.

SOURCE
Guidelines · 5.1 Rubric Fundamentals
Guidelines · 5.4 Category & Evaluation Target

SEE EXAMPLE [19](#)

Model A has to fail half the rubric

A hard client requirement: the Model A trajectory clearly fails at least 50% of the final rubric score, with failures that materially affect task completion and the user experience. Model A is rated on the subjective block too, so both blocks count.

Failed weight · positives rated Not Present plus negatives rated Present, signs ignored
Divided by · the weight of every criterion in both blocks, signs ignored

Why The difficulty comes from the idea. If the outcome is reached too easily, increase the complexity of the scenario; heavier weights on the same rows change nothing.

SOURCE
Guidelines · Hard Client Requirements for Every Task
Guidelines · 8.5 Rating Against both Models and Justifications

SEE EXAMPLE [20](#)

Criteria a correct run can pass

A criterion has to pass a correct run, fail a wrong one, carry the right Category and Evaluation Target, and count once. If your own golden fails one, check the criterion first: it is usually the one that is broken.

The golden passes the complete rubric

Your golden has to pass every objective criterion. Anything it fails is a broken criterion, not a broken golden. The usual cause is a rule only the Desired Outcome states: the agent only ever sees the prompts and the inputs.

Why A criterion a correct run cannot satisfy is Major. Check before you complete Leg B, while a cleanup turn can still fix the golden if the golden is what is wrong.

SOURCE
Guidelines · 5.7 Final Rubric Checklist
Guidelines · 4. Valid vs. Invalid Model Failures

SEE EXAMPLES [21](#) AND [22](#)

MOST OFTEN BROKEN

A criterion is as strict as its source

Grade only what the scenario settles objectively. A criterion accepts every valid implementation and rejects the invalid ones: no exact figure the source leaves open, no one answer where several are defensible, no single route where two sources carry the value.

Why Too specific rejects valid runs; too loose accepts wrong ones. Both are Moderate. If you are unsure which answer is right, the GTFA is not settled.

SOURCE
Guidelines · 5.1 Rubric Fundamentals
QC spec appendix · Rubric Quality Definitions · Overfitting and Underfitting

SEE EXAMPLES [23](#), [24](#) AND [25](#)

Criteria a correct run can pass

A criterion has to pass a correct run, fail a wrong one, carry the right Category and Evaluation Target, and count once. If your own golden fails one, check the criterion first: it is usually the one that is broken.

Category and Evaluation Target

The target is mechanical, set by where the grader looks. The category says what kind of check it is, and Agent Behavior always sits on Trajectory.

- Final Answer Artifact** · a file the agent created or modified
- State Change** · a record changed in the service, through snapshots.json
- User-Facing Message** · the final assistant message
- Trajectory** · the agent's reasoning path

Why Both fields shape how the rubric data reaches the client. In the subjective block Task Completion is the only valid category: anything the prompt asked for is objective.

SOURCE
Guidelines · 5.4 Category & Evaluation Target
Guidelines · 8.4 Categories and Evaluation Targets for Subjective Rubrics

SEE EXAMPLES [26](#) AND [27](#)

Each piece of evidence is graded once

Two criteria decided by the same evidence are one check counted twice: one swallowed by a broader one, a negative that mirrors a positive, or a subjective row restating an objective value.

Why A single slip then costs the run twice. A check in the wrong block is moved to the right one, never duplicated across both.

SOURCE
QC spec appendix · Rubric Quality Definitions · Overlapping/Redundant Criteria
QC spec appendix · Rubric Quality Definitions · Block Bleed

SEE EXAMPLE [28](#)

The subjective block

Ten or more criteria, no more than 30% of the weight, each grading a property of the render rather than your golden's wording, and each naming its file when the task ships more than one. Check all four before you submit.

Anchor each criterion to a named file

Each subjective criterion points to one identifiable element and one visible property, and it has to be rateable from itself and its target alone. When the task ships more than one file, the filename is part of telling the reviewer where to look.

Why The target only says Final Answer Artifact. With two deliverables, “the table” identifies nothing, and two graders can rate the same run against different files.

SOURCE
Guidelines · 5.1 Rubric Fundamentals
Guidelines · 8.1 Writing Subjective Rubrics

SEE EXAMPLE [29](#)

Ten or more criteria, at most 30% of the weight

The subjective block expects 10 or more criteria, from a deliverable rich enough to justify them, never padded to hit the count. The block is capped at 30% of the task's total weight.

Why Ten is a floor, not a target. The appendix lists six dimensions worth covering, from layout and legibility to register for the stated audience, and a block clustered on one leaves the rest of the render ungraded.

SOURCE
Guidelines · 8.1 Writing Subjective Rubrics
QC spec appendix · Multi-Turn Authoring Standards · Writing Subjective Rubrics

SEE EXAMPLES [30](#) AND [31](#)

Grade a property of the render

Subjective criteria grade the render side the prompt never asked for: layout, hierarchy, spacing, readability, structure, consistency. Name one observable property, never the wording one build happened to use.

Why Anything the prompt fixed is objective only. A criterion quoting your golden's headings rejects every valid page worded differently: that is Overfitting.

SOURCE
QC spec appendix · Multi-Turn Authoring Standards · Writing Subjective Rubrics
Guidelines · 8.1 Writing Subjective Rubrics

SEE EXAMPLE [32](#)

Closing the task

The golden is a deliverable, not a run you stopped. Check every value against its source and every revision for wording it left behind, then run the close out before you submit.

Every golden claim is grounded in the inputs

The golden is the ideal response, and every factual claim in it is grounded in the actual inputs: images opened, videos watched, PDFs read. That includes values no criterion grades.

Why Subjective ratings and pairwise scoring are graded relative to the golden, so a wrong value in it is scored as correct. QC fails a golden asserting what its inputs do not support.

SOURCE
QC spec · Golden Solution - Artifact Completeness Guidelines · 7. Golden Solution - Leg B

SEE EXAMPLES [33](#) [↗] AND [34](#) [↗]

A revision leaves nothing stale

A revision turn changes work already delivered. The golden shows the edit applied to that work: earlier content kept, the change visible, nothing left stale that depended on it.

Why The spec's own failing case: a header still saying five when three remain. The edit touched the table and not the parts that depend on it.

SOURCE
QC spec · Multi-Turn - Revision Turn Handling Guidelines · 6.3 Modifiers

SEE EXAMPLE [35](#) [↗]

The golden ships finished and complete

golden/ ships finished artifacts only: every file the prompts name, spelled as they spell it, and nothing describing how the work was made. The preferred run is starred.

Why The judge receives the artifacts, never the trajectory, so a script or build note left in the folder leaks the solution path. A missing star costs points on its own.

SOURCE
QC spec · Golden Solution - Trajectory Exclusion Guidelines · 7.1 What Changes from Leg A

SEE EXAMPLES [36](#) [↗] AND [37](#) [↗]

The checks, section by section

If you read one slide twice, read this one. One check per standard in this deck, and every one of them a real task failed.

1. THE FOUNDATIONS EVERY TASK MUST GET RIGHT

- 1 The deliverable clears its floor on the complexity bar
- 2 The universe returns every record your rules point at

2. WHAT YOU MAY NOT TELL MODEL A OR MODEL B

- 1 Both legs: no turn you type gives Model A or Model B a value, count, position or conclusion
- 2 Both legs: the opening prompt leaves thresholds, rules and formats to the input files
- 3 Both legs: deferred files are neutrally named, and nothing in the input folder previews their turn
- 4 Leg A: any correction is punctual, comes up naturally and leaves the 50% failure threshold intact
- 5 Leg B: every hint to Model B points at the area, never at the answer or the exact file

3. MILESTONES

- 1 Every modifier is one of the four; Assets Delivered lists what each Leg A turn handed over
- 2 Each milestone carries one checkable clause per requirement of its Leg A turn, and only its own
- 3 Every Leg A turn has a milestone, and no requirement appears twice

4. A RUBRIC THAT SEPARATES TWO RUNS

- 1 About a quarter negatives, each a trap the scenario really sets
- 2 Every ask has a criterion, and every criterion an ask
- 3 Each weight matches the axes its criterion exercises
- 4 Five trajectory criteria at most, each something only the run shows
- 5 Model A fails at least half the weight, both blocks counted

5. CRITERIA A CORRECT RUN CAN PASS

- 1 The golden passes every objective criterion, checked before you complete Leg B
- 2 Every criterion is as strict as its source, no more and no less
- 3 Agent Behavior sits on Trajectory; every subjective row is Task Completion
- 4 No two criteria are decided by the same evidence

6. THE SUBJECTIVE BLOCK

- 1 Every subjective criterion names the file it grades
- 2 Ten or more subjective criteria, at most 30% of the weight
- 3 Subjective criteria grade properties, never your own wording

7. CLOSING THE TASK

- 1 Every golden value matches its source, everywhere it appears
- 2 Nothing in the golden still describes the state before a revision
- 3 The archive holds only the named files, and the golden run is starred

AFTER THIS COURSE

What happens from here is simple

You have seen the mistakes that keep costing tasks and project access. Nobody is expected to be perfect, but once an issue has been identified and explained, it is expected to be corrected, not repeated.



These are not hidden expectations anymore: the standards, the real examples and how to meet them are all in this course.

The goal is not to catch you making mistakes. It is to make sure the same preventable mistakes stop happening.

Thank you for your engagement and cooperation.